AI ORGANISATION

Pipeline Run Report

IDEA

Build a B2B mobile-first pharmacy ordering app where a retail pharmacist (the retailer) can browse medicines from multiple distributors, place orders, manage a cart, see active and closed orders, and view their store profile. After login, the retailer lands on a 4-tab dashboard: Home (offers carousel + medicine search + quick links to Distributors / Schemes / Generic Medicines / Scan Prescription / Outstanding Payments), Orders (Active / Closed sub-tabs), Cart (line items with distributor + price), Profile (name, license number, favourites, store location).

Decision: WROTE_FILES

TIMESTAMP	2026-05-02T19:09:20.368Z
TOTAL TIME	2 ms across 9 step(s)
VALIDATION	valid · 6 feature(s), 10 task(s)
SELECTED TASK	t-data-1 (developer, 4h) -> f-data-spine
QA	PASS
CYBERSECURITY	GO
FILES WRITTEN	2 file(s), 18,238 bytes

This report captures the full agent chain (CEO -> CTO -> Engineering Manager -> Developer -> QA -> Cybersecurity). Each agent's complete output is included in the per-agent sections.

Steps

#	Step	Status	Duration	Notes
1	ceo	ok	0 ms	
2	cto	ok	0 ms	
3	engineering-manager	ok	0 ms	
4	validation	ok	1 ms	6 features, 10 tasks
5	task-selection	ok	0 ms	picked t-data-1 (developer, 4h) under f-data-spine
6	developer	ok	0 ms	
7	qa	ok	0 ms	decision=PASS
8	cybersecurity	ok	0 ms	decision=GO
9	file-writer	ok	1 ms	2 file(s), 18238 bytes

Validation

Valid — 6 feature(s), 10 task(s).

Selected task

Task: t-data-1 — Implement the in-memory data spine: lib/types.ts (TypeScript types for Retailer, Distributor, Medicine, Offer, CartItem, Order, OrderItem, OutstandingPayment, OrderStatus, derived Money helpers) and lib/db/store.ts (Map-backed stores seeded at module import with 1 pilot retailer, 4 distributors, 20 medicines spanning the four distributors, 3 active offers, 2 outstanding-payment rows; CRUD helpers - getRetailer, listDistributors, listOffers, searchMedicines(q), getMedicine(id), getCart(retailerId), addCartItem, removeCartItem, clearCart, listOrders(retailerId, statusGroup), createOrdersFromCart(retailerId), getOutstandingForRetailer(retailerId)).

Assignee: developer

Estimate: 4 h

Feature: f-data-spine — In-memory data spine + shared types

Files written

2 file(s), 18,238 bytes total.

Path	Bytes
lib/db/store.ts	15,207
lib/types.ts	3,031

CEO

Mission: Let an independent retail pharmacist place orders to every medicine distributor they buy from in one mobile-first app, replacing the WhatsApp + phone-call ordering loop they live in today.

OKRS

- Ship a retailer-only build with login and a 4-tab dashboard (Home, Orders, Cart, Profile) that supports the full search -> add-to-cart -> place-order -> see-in-active-orders loop within 14 days
- Convert 50% of the 6 pilot retailers from their current channel to at least one in-app order in the first 30 days post-launch
- Surface an outstanding-payments tile on the Home tab so retailers can see distributor-wise balance without calling, reducing reconciliation calls per retailer by 70% in the same window

PRIORITIES

- Stand up the authenticated 4-tab dashboard shell (mock retailer login + tab navigation) so every later task has a place to land in the UI
- Build the Home tab: search bar at the top, offers carousel, and the 5 quick-link tiles (Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments)
- Build the Cart and Orders tabs end-to-end: cart line items with distributor + total, place-order action, and Active/Closed sub-tabs in Orders
- Build the Profile tab as a read-only retailer card (name, license number, favourites count, store location) plus the outstanding-payments stub it links to from Home

DELEGATION

CTO: Build a mobile-first Next.js 14 (App Router) + Tailwind webapp. Backend is Next.js Route Handlers in the same project; demo data lives in in-memory Maps seeded at boot (one pilot retailer, four distributors, ~20 medicines, three offers). Authentication is a mock cookie-based session: POST /api/auth/login takes a license number and sets a pharmacy-session cookie, middleware guards /dashboard/*. No real distributor integration, no real payments, no Postgres this cycle. Ship by end of day 8 with a runnable end-to-end loop: login -> search a medicine -> add to cart -> place order -> see it under Orders/Active -> view Profile.

CMO: Skip this cycle. Pilot is direct sign-up with the 6 retailers we already have a relationship with; no paid acquisition until cycle 2.

CFO: Back-of-envelope only: distributor commission percent x average order value x monthly orders per retailer. Re-validate before cycle 2; no spend this cycle other than the existing engineering team.

CPO: Own the product copy: empty states (no orders, empty cart, no offers), the 'outstanding payments' tile heading and per-distributor row, the 5 quick-link tile labels and supporting microcopy, and the order-status vocabulary (Placed / Acknowledged / Out for delivery / Delivered / Cancelled). Hand the copy sheet to CTO by end of day 2.

FULL CEO OUTPUT (JSON)


```
{
  "mission": "Let an independent retail pharmacist place orders to every medicine distributor
they buy from in one mobile-first app, replacing the WhatsApp + phone-call ordering loop they
live in today.",
  "okrs": [
    "Ship a retailer-only build with login and a 4-tab dashboard (Home, Orders, Cart, Profile)
that supports the full search -> add-to-cart -> place-order -> see-in-active-orders loop within
14 days",
    "Convert 50% of the 6 pilot retailers from their current channel to at least one in-app order
in the first 30 days post-launch",
```

```
    "Surface an outstanding-payments tile on the Home tab so retailers can see distributor-wise balance without calling, reducing reconciliation calls per retailer by 70% in the same window"
  ],
  "priorities": [
    "Stand up the authenticated 4-tab dashboard shell (mock retailer login + tab navigation) so every later task has a place to land in the UI",
    "Build the Home tab: search bar at the top, offers carousel, and the 5 quick-link tiles (Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments)",
    "Build the Cart and Orders tabs end-to-end: cart line items with distributor + total, place-order action, and Active/Closed sub-tabs in Orders",
    "Build the Profile tab as a read-only retailer card (name, license number, favourites count, store location) plus the outstanding-payments stub it links to from Home"
  ],
  "delegation": {
    "cto": "Build a mobile-first Next.js 14 (App Router) + Tailwind webapp. Backend is Next.js Route Handlers in the same project; demo data lives in in-memory Maps seeded at boot (one pilot retailer, four distributors, ~20 medicines, three offers). Authentication is a mock cookie-based session: POST /api/auth/login takes a license number and sets a pharmacy-session cookie, middleware guards /dashboard/*. No real distributor integration, no real payments, no Postgres this cycle. Ship by end of day 8 with a runnable end-to-end loop: login -> search a medicine -> add to cart -> place order -> see it under Orders/Active -> view Profile.",
    "cmo": "Skip this cycle. Pilot is direct sign-up with the 6 retailers we already have a relationship with; no paid acquisition until cycle 2.",
    "cfo": "Back-of-envelope only: distributor commission percent x average order value x monthly orders per retailer. Re-validate before cycle 2; no spend this cycle other than the existing engineering team.",
    "cpo": "Own the product copy: empty states (no orders, empty cart, no offers), the 'outstanding payments' tile heading and per-distributor row, the 5 quick-link tile labels and supporting microcopy, and the order-status vocabulary (Placed / Acknowledged / Out for delivery / Delivered / Cancelled). Hand the copy sheet to CTO by end of day 2."
  }
}
```

CTO

ARCHITECTURE

Frontend: Next.js 14 (App Router) + Tailwind CSS, mobile-first responsive layout. Authenticated /dashboard route renders a fixed bottom tab bar (Home / Orders / Cart / Profile) on small screens that promotes to a side rail at md+. Client components only where state is needed (search input, cart updates, tab toggles); everything else is a server component reading from the in-process data layer.

Backend: Next.js Route Handlers under app/api/* in the same Next project (no separate Node service). Auth uses a signed cookie set via Next's cookies() API; middleware at the project root guards /dashboard/* and every /api/* route except /api/auth/login. All handlers are typed end-to-end via shared zod schemas in lib/api/contracts.ts.

Database: In-memory storage via lib/db/store.ts (Map<string, Record>), seeded at module load with 1 pilot retailer, 4 distributors, ~20 medicines, 3 offers, and 2 outstanding-payment rows. The schema is shaped to match a future Postgres + Drizzle migration so the same TypeScript types survive the swap; no migration this cycle.

Infrastructure: Single Vercel deploy (frontend + Route Handlers behind one origin). Static assets via Vercel CDN. No external services this cycle. Future cycles: Neon Postgres for persistence, per-distributor HTTP integration via a simple fanout, and S3-compatible storage for prescription scans.

API CONTRACTS (11)

- POST /api/auth/login — Pilot login: takes a retailer license number, looks it up in the in-memory store, sets the pharmacy-session cookie (HttpOnly, SameSite=Lax) and returns the retailer summary. No password this cycle (single pilot retailer per device).
- POST /api/auth/logout — Clears the pharmacy-session cookie.
- GET /api/medicines/search — Case-insensitive substring search over medicine name, brand, and generic name. Empty q returns the full catalogue (used by the search results page when a retailer lands without a query). Each result carries the distributor it ships from so the UI can render the price + supplier chip.
- GET /api/distributors — List all distributors the retailer can buy from. Used by the Home tab quick-link and the cart line items.
- GET /api/offers — List of currently-active offers shown in the Home tab carousel. Returned in sortOrder. Each entry references the distributor running the offer.
- GET /api/cart — Returns the current retailer's cart, grouped by distributor with a per-distributor subtotal and a grand total. Drives the Cart tab.
- POST /api/cart/items — Add a medicine to the current retailer's cart, or increment the qty if it is already there. Validates that the medicineId exists. Returns the updated cart payload (same shape as GET /api/cart) so the Cart tab can render without a second round-trip.
- DELETE /api/cart/items/{medicineId} — Remove a medicine from the current retailer's cart. Returns the updated cart payload.
- POST /api/orders — Place orders from the current retailer's cart. The cart is fanned out into one order per distributor (so the retailer's UI can show each supplier's order separately under Orders > Active). Cart is cleared on success.
- GET /api/orders — List the retailer's orders, filtered by status group. status=active returns placed | acknowledged | out_for_delivery; status=closed returns delivered | cancelled. Sorted newest-first.
- GET /api/profile — Returns the current retailer's profile card plus the outstanding-payments breakdown shown on the Home tab tile.

DATABASE SCHEMA (8)

- **retailers**
  - id: string primary key (uuid in production)
  - name: string not null (display name shown on Profile)
  - owner_name: string not null
  - license_number: string not null unique (used as login id this cycle)
  - store_name: string not null
  - store_address: string not null
  - phone: string not null
  - email: string not null
  - gstin: string not null
  - favourites_medicine_ids: string[] not null default []
  - created_at: timestamp not null
- **distributors**
  - id: string primary key
  - name: string not null
  - region: string not null
  - supplier_code: string not null unique
  - contact_phone: string not null
  - contact_email: string not null
  - rating: number not null (0-5, decimal)
  - created_at: timestamp not null
- **medicines**
  - id: string primary key
  - name: string not null
  - brand: string not null
  - manufacturer: string not null
  - generic_name: string not null
  - distributor_id: string not null references distributors(id)
  - mrp_paise: integer not null (>=0)
  - selling_price_paise: integer not null (>=0)
  - scheme: string nullable (e.g. '10+1 free' or null)
  - pack_size: string not null (e.g. '10 tablets', '100 ml syrup')
  - hsn_code: string not null
  - created_at: timestamp not null
- **offers**
  - id: string primary key
  - title: string not null
  - description: string not null
  - distributor_id: string not null references distributors(id)
  - banner_label: string not null (short text shown on the gradient banner)
  - valid_until: timestamp not null
  - sort_order: integer not null (lower = earlier in the carousel)
  - created_at: timestamp not null
- **cart_items**
  - id: string primary key
  - retailer_id: string not null references retailers(id)
  - medicine_id: string not null references medicines(id)
  - distributor_id: string not null references distributors(id) (denormalised from medicines for easy grouping)
  - qty: integer not null (>=1)
  - unit_price_paise: integer not null (frozen at add-to-cart time)
  - added_at: timestamp not null

    - unique(retailer_id, medicine_id)
- **orders**
    - id: string primary key
    - retailer_id: string not null references retailers(id)
    - distributor_id: string not null references distributors(id)
    - status: string not null (one of: placed | acknowledged | out_for_delivery | delivered | cancelled)
    - placed_at: timestamp not null
    - expected_delivery: timestamp nullable
    - total_paise: integer not null (>=0)
    - item_count: integer not null (>=1)
- **order_items**
    - id: string primary key
    - order_id: string not null references orders(id)
    - medicine_id: string not null references medicines(id)
    - qty: integer not null (>=1)
    - unit_price_paise: integer not null
    - line_total_paise: integer not null
- **outstanding_payments**
    - id: string primary key
    - retailer_id: string not null references retailers(id)
    - distributor_id: string not null references distributors(id)
    - amount_due_paise: integer not null (>=0)
    - last_updated_at: timestamp not null
    - unique(retailer_id, distributor_id)

RISKS (6)

- security: mock cookie auth (single shared session secret, no password, no multi-device invalidation) is intentional for the pilot but unsafe for production; flag for a real auth task in cycle 2
- compliance: pharmacy ordering is regulated (drug schedules, Schedule H1/X tracking, narcotics audit trail). This cycle ignores schedule classification and prescription validation; both are mandatory before any non-pilot rollout
- data loss: in-memory Maps reset on every process restart. Acceptable for a demo, blocking for a customer pilot
- scaling: medicine search is a linear scan over the in-memory store. Fine for ~20 medicines; needs Postgres trigram index or Meilisearch beyond ~1000 SKUs
- mobile UX: 'Scan Prescription' is a placeholder tile this cycle; barcode + OCR scan needs a native shell or WebRTC + a vision model and is out of scope
- outstanding-payments accuracy: the per-distributor balance is currently a static seed; integration with each distributor's ledger is a downstream task and the tile must be labelled as 'last reconciled at <ts>' to avoid retailer confusion

FULL CTO OUTPUT (JSON)


```
{
  "architecture": {
    "frontend": "Next.js 14 (App Router) + Tailwind CSS, mobile-first responsive layout. Authenticated /dashboard route renders a fixed bottom tab bar (Home / Orders / Cart / Profile) on small screens that promotes to a side rail at md+. Client components only where state is needed (search input, cart updates, tab toggles); everything else is a server component reading from the in-process data layer.",
    "backend": "Next.js Route Handlers under app/api/* in the same Next project (no separate Node service). Auth uses a signed cookie set via Next's cookies() API; middleware at the project root guards /dashboard/* and every /api/* route except /api/auth/login. All handlers are typed end-to-end via shared zod schemas in lib/api/contracts.ts.",
    "database": "In-memory storage via lib/db/store.ts (Map<string, Record>), seeded at module load with 1 pilot retailer, 4 distributors, ~20 medicines, 3 offers, and 2 outstanding-payment rows. The schema is shaped to match a future Postgres + Drizzle migration so the same TypeScript types survive the swap; no migration this cycle.",
    "infrastructure": "Single Vercel deploy (frontend + Route Handlers behind one origin). Static assets via Vercel CDN. No external services this cycle. Future cycles: Neon Postgres for
```

```
persistence, per-distributor HTTP integration via a simple fanout, and S3-compatible storage for
prescription scans."
  },
  "apiContracts": [
    {
      "endpoint": "/api/auth/login",
      "method": "POST",
      "description": "Pilot login: takes a retailer license number, looks it up in the in-memory
store, sets the pharmacy-session cookie (HttpOnly, SameSite=Lax) and returns the retailer
summary. No password this cycle (single pilot retailer per device).",
      "request": {
        "licenseNumber": "string"
      },
      "response": {
        "retailer": {
          "id": "string",
          "name": "string",
          "storeName": "string",
          "licenseNumber": "string"
        }
      }
    },
    {
      "endpoint": "/api/auth/logout",
      "method": "POST",
      "description": "Clears the pharmacy-session cookie.",
      "request": {},
      "response": {
        "ok": "true"
      }
    },
    {
      "endpoint": "/api/medicines/search",
      "method": "GET",
      "description": "Case-insensitive substring search over medicine name, brand, and generic
name. Empty q returns the full catalogue (used by the search results page when a retailer lands
without a query). Each result carries the distributor it ships from so the UI can render the
price + supplier chip.",
      "request": {
        "q": "string (optional)"
      },
      "response": {
        "results": [
          {
            "id": "string",
            "name": "string",
            "brand": "string",
            "genericName": "string",
            "distributor": {
              "id": "string",
              "name": "string"
            },
            "mrpPaise": "integer",
            "sellingPricePaise": "integer",
            "scheme": "string | null",
            "packSize": "string"
          }
        ]
      }
    },
    {
      "endpoint": "/api/distributors",
      "method": "GET",
      "description": "List all distributors the retailer can buy from. Used by the Home tab quick-
link and the cart line items.",
      "request": {},
      "response": {
        "distributors": [
          {
            "id": "string",
            "name": "string",
            "region": "string",
            "rating": "number"
          }
        ]
```

```
      }
    },
    {
      "endpoint": "/api/offers",
      "method": "GET",
      "description": "List of currently-active offers shown in the Home tab carousel. Returned in sortOrder. Each entry references the distributor running the offer.",
      "request": {},
      "response": {
        "offers": [
          {
            "id": "string",
            "title": "string",
            "description": "string",
            "distributor": {
              "id": "string",
              "name": "string"
            },
            "validUntil": "ISO-8601 string"
          }
        ]
      }
    },
    {
      "endpoint": "/api/cart",
      "method": "GET",
      "description": "Returns the current retailer's cart, grouped by distributor with a per-distributor subtotal and a grand total. Drives the Cart tab.",
      "request": {},
      "response": {
        "groups": [
          {
            "distributor": {
              "id": "string",
              "name": "string"
            },
            "items": [
              {
                "medicineId": "string",
                "name": "string",
                "brand": "string",
                "qty": "integer",
                "unitPricePaise": "integer",
                "lineTotalPaise": "integer"
              }
            ],
            "subtotalPaise": "integer"
          }
        ],
        "grandTotalPaise": "integer",
        "itemCount": "integer"
      }
    },
    {
      "endpoint": "/api/cart/items",
      "method": "POST",
      "description": "Add a medicine to the current retailer's cart, or increment the qty if it is already there. Validates that the medicineId exists. Returns the updated cart payload (same shape as GET /api/cart) so the Cart tab can render without a second round-trip.",
      "request": {
        "medicineId": "string",
        "qty": "integer (>=1, default 1)"
      },
      "response": {
        "groups": [
          {
            "distributor": {
              "id": "string",
              "name": "string"
            },
            "items": [
              {
                "medicineId": "string",
                "name": "string",
                "brand": "string",
```

```
              "qty": "integer",
              "unitPricePaise": "integer",
              "lineTotalPaise": "integer"
            }
          ],
          "subtotalPaise": "integer"
        }
      ],
      "grandTotalPaise": "integer",
      "itemCount": "integer"
    }
  },
  {
    "endpoint": "/api/cart/items/{medicineId}",
    "method": "DELETE",
    "description": "Remove a medicine from the current retailer's cart. Returns the updated
cart payload.",
    "request": {
      "medicineId": "string (path param)"
    },
    "response": {
      "groups": [
        {
          "distributor": {
            "id": "string",
            "name": "string"
          },
          "items": [
            {
              "medicineId": "string",
              "name": "string",
              "brand": "string",
              "qty": "integer",
              "unitPricePaise": "integer",
              "lineTotalPaise": "integer"
            }
          ],
          "subtotalPaise": "integer"
        }
      ],
      "grandTotalPaise": "integer",
      "itemCount": "integer"
    }
  },
  {
    "endpoint": "/api/orders",
    "method": "POST",
    "description": "Place orders from the current retailer's cart. The cart is fanned out into
one order per distributor (so the retailer's UI can show each supplier's order separately under
Orders > Active). Cart is cleared on success.",
    "request": {},
    "response": {
      "orderIds": [
        "string"
      ],
      "orderCount": "integer"
    }
  },
  {
    "endpoint": "/api/orders",
    "method": "GET",
    "description": "List the retailer's orders, filtered by status group. status=active returns
placed | acknowledged | out_for_delivery; status=closed returns delivered | cancelled. Sorted
newest-first.",
    "request": {
      "status": "active | closed"
    },
    "response": {
      "orders": [
        {
          "id": "string",
          "distributor": {
            "id": "string",
            "name": "string"
          },
```

```
            "status": "placed | acknowledged | out_for_delivery | delivered | cancelled",
            "placedAt": "ISO-8601 string",
            "expectedDelivery": "ISO-8601 string | null",
            "itemCount": "integer",
            "totalPaise": "integer"
          }
        ]
      }
    },
    {
      "endpoint": "/api/profile",
      "method": "GET",
      "description": "Returns the current retailer's profile card plus the outstanding-payments
breakdown shown on the Home tab tile.",
      "request": {},
      "response": {
        "retailer": {
          "id": "string",
          "name": "string",
          "ownerName": "string",
          "licenseNumber": "string",
          "storeName": "string",
          "storeAddress": "string",
          "phone": "string",
          "email": "string",
          "gstin": "string",
          "favouritesCount": "integer"
        },
        "outstanding": {
          "totalPaise": "integer",
          "perDistributor": [
            {
              "distributor": {
                "id": "string",
                "name": "string"
              },
              "amountPaise": "integer"
            }
          ]
        }
      }
    }
  ],
  "databaseSchema": [
    {
      "table": "retailers",
      "fields": [
        "id: string primary key (uuid in production)",
        "name: string not null (display name shown on Profile)",
        "owner_name: string not null",
        "license_number: string not null unique (used as login id this cycle)",
        "store_name: string not null",
        "store_address: string not null",
        "phone: string not null",
        "email: string not null",
        "gstin: string not null",
        "favourites_medicine_ids: string[] not null default []",
        "created_at: timestamp not null"
      ]
    },
    {
      "table": "distributors",
      "fields": [
        "id: string primary key",
        "name: string not null",
        "region: string not null",
        "supplier_code: string not null unique",
        "contact_phone: string not null",
        "contact_email: string not null",
        "rating: number not null (0-5, decimal)",
        "created_at: timestamp not null"
      ]
    },
    {
      "table": "medicines",
```

```
      "fields": [
        "id: string primary key",
        "name: string not null",
        "brand: string not null",
        "manufacturer: string not null",
        "generic_name: string not null",
        "distributor_id: string not null references distributors(id)",
        "mrp_paise: integer not null (>=0)",
        "selling_price_paise: integer not null (>=0)",
        "scheme: string nullable (e.g. '10+1 free' or null)",
        "pack_size: string not null (e.g. '10 tablets', '100 ml syrup')",
        "hsn_code: string not null",
        "created_at: timestamp not null"
      ]
    },
    {
      "table": "offers",
      "fields": [
        "id: string primary key",
        "title: string not null",
        "description: string not null",
        "distributor_id: string not null references distributors(id)",
        "banner_label: string not null (short text shown on the gradient banner)",
        "valid_until: timestamp not null",
        "sort_order: integer not null (lower = earlier in the carousel)",
        "created_at: timestamp not null"
      ]
    },
    {
      "table": "cart_items",
      "fields": [
        "id: string primary key",
        "retailer_id: string not null references retailers(id)",
        "medicine_id: string not null references medicines(id)",
        "distributor_id: string not null references distributors(id) (denormalised from medicines
for easy grouping)",
        "qty: integer not null (>=1)",
        "unit_price_paise: integer not null (frozen at add-to-cart time)",
        "added_at: timestamp not null",
        "unique(retailer_id, medicine_id)"
      ]
    },
    {
      "table": "orders",
      "fields": [
        "id: string primary key",
        "retailer_id: string not null references retailers(id)",
        "distributor_id: string not null references distributors(id)",
        "status: string not null (one of: placed | acknowledged | out_for_delivery | delivered |
cancelled)",
        "placed_at: timestamp not null",
        "expected_delivery: timestamp nullable",
        "total_paise: integer not null (>=0)",
        "item_count: integer not null (>=1)"
      ]
    },
    {
      "table": "order_items",
      "fields": [
        "id: string primary key",
        "order_id: string not null references orders(id)",
        "medicine_id: string not null references medicines(id)",
        "qty: integer not null (>=1)",
        "unit_price_paise: integer not null",
        "line_total_paise: integer not null"
      ]
    },
    {
      "table": "outstanding_payments",
      "fields": [
        "id: string primary key",
        "retailer_id: string not null references retailers(id)",
        "distributor_id: string not null references distributors(id)",
        "amount_due_paise: integer not null (>=0)",
        "last_updated_at: timestamp not null",
```

```
        "unique(retailer_id, distributor_id)"
      ]
    }
  ],
  "risks": [
    "security: mock cookie auth (single shared session secret, no password, no multi-device invalidation) is intentional for the pilot but unsafe for production; flag for a real auth task in cycle 2",
    "compliance: pharmacy ordering is regulated (drug schedules, Schedule H1/X tracking, narcotics audit trail). This cycle ignores schedule classification and prescription validation; both are mandatory before any non-pilot rollout",
    "data loss: in-memory Maps reset on every process restart. Acceptable for a demo, blocking for a customer pilot",
    "scaling: medicine search is a linear scan over the in-memory store. Fine for ~20 medicines; needs Postgres trigram index or Meilisearch beyond ~1000 SKUs",
    "mobile UX: 'Scan Prescription' is a placeholder tile this cycle; barcode + OCR scan needs a native shell or WebRTC + a vision model and is out of scope",
    "outstanding-payments accuracy: the per-distributor balance is currently a static seed; integration with each distributor's ledger is a downstream task and the tile must be labelled as 'last reconciled at <ts>' to avoid retailer confusion"
  ]
}
```

Engineering Manager

FEATURES (6)

- f-shell-auth (high) — App shell + mock retailer auth
- f-data-spine (high) — In-memory data spine + shared types
- f-home-tab (high) — Home tab
- f-orders-tab (high) — Orders tab + place-order API
- f-cart-tab (high) — Cart tab + cart APIs
- f-profile-tab (medium) — Profile tab

TASKS (10)

- t-data-1 -> f-data-spine · developer · 4h - Implement the in-memory data spine: lib/types.ts (TypeScript types for Retailer, Distributor, Medicine, Offer, CartItem, Order, OrderItem, OutstandingPayment, OrderStatus, derived Money helpers) and lib/db/store.ts (Map-backed stores seeded at module import with 1 pilot retailer, 4 distributors, 20 medicines spanning the four distributors, 3 active offers, 2 outstanding-payment rows; CRUD helpers - getRetailer, listDistributors, listOffers, searchMedicines(q), getMedicine(id), getCart(retailerId), addCartItem, removeCartItem, clearCart, listOrders(retailerId, statusGroup), createOrdersFromCart(retailerId), getOutstandingForRetailer(retailerId)).
- t-shell-1 -> f-shell-auth · developer · 4h - Set up the Next.js 14 + Tailwind project skeleton: package.json (next 14.2, react 18, tailwind 3, typescript 5, vitest), tsconfig.json with the @/* alias, next.config.mjs, postcss.config.mjs, tailwind.config.ts (mobile-first content globs), app/globals.css with the Tailwind directives + base background, app/layout.tsx with the html/body shell, and the public login page at app/page.tsx that posts to /api/auth/login. No auth handler yet (lands in t-shell-2); the page just calls fetch and redirects on success.
- t-shell-2 -> f-shell-auth · developer · 4h - Implement the mock auth: POST /api/auth/login (look up the retailer by license number against the in-memory store, set the pharmacy-session cookie HttpOnly+SameSite=Lax with a 30-day max-age), POST /api/auth/logout (clear the cookie), middleware.ts that gates /dashboard/* and every /api/* route except /api/auth/login (returns 401 JSON if missing cookie), a small lib/auth/session.ts helper that reads the current retailerId from the request cookies, and the authenticated app/dashboard/layout.tsx that renders the 4-tab bottom navigation (Home / Orders / Cart / Profile) and a stub app/dashboard/page.tsx that redirects to /dashboard/home.
- t-home-1 -> f-home-tab · developer · 3h - Implement the read APIs that drive the Home tab: GET /api/medicines/search (case-insensitive substring over name + brand + generic, returns each result with its distributor), GET /api/distributors (id/name/region/rating list), GET /api/offers (active offers in sortOrder, each with its distributor). All three use the auth-guarded session helper to identify the retailer (search results may carry per-retailer favourites later) and return 401 if the cookie is missing.
- t-home-2 -> f-home-tab · developer · 4h - Build the Home tab page at app/dashboard/home/page.tsx: greeting line with the retailer name, a search bar at the top that links to /dashboard/search?q=..., a horizontally scrollable offers carousel reading from GET /api/offers, the 5 quick-link tiles (Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments), and an outstanding-payments summary tile that calls GET /api/profile and shows the total + per-distributor breakdown. Mobile-first Tailwind layout, snap-scroll on the carousel.
- t-cart-1 -> f-cart-tab · developer · 3h - Implement the cart APIs: GET /api/cart (returns the cart grouped by distributor with subtotals + grand total + itemCount, derived from the in-memory store), POST /api/cart/items (validates medicineId exists, increments qty if already present, returns the same grouped payload), DELETE /api/cart/items/[medicineId] (removes the line, returns the same grouped payload). All three resolve the retailer via the session helper and 401 without it.
- t-cart-2 -> f-cart-tab · developer · 3h - Build the Cart tab page at app/dashboard/cart/page.tsx: groups the cart by distributor (heading per supplier with subtotal), each line shows name + brand + qty stepper + remove button + line total, sticky footer with the grand total and a Place Order button

that POSTs to /api/orders and on success routes to /dashboard/orders?status=active. Empty-state copy when the cart is empty.

- t-orders-1 -> f-orders-tab · developer · 3h - Implement the orders APIs: GET /api/orders?status=active|closed (active = placed | acknowledged | out_for_delivery, closed = delivered | cancelled, sorted newest-first), POST /api/orders (reads the current retailer's cart, fans out into one order per distinct distributorId, copies cart_items into order_items, clears the cart, returns the new orderIds + count). 401 without the session cookie.
- t-orders-2 -> f-orders-tab · developer · 3h - Build the Orders tab page at app/dashboard/orders/page.tsx with Active and Closed sub-tabs (segmented control). Each sub-tab calls GET /api/orders with the matching status group and renders order cards (distributor name, status pill with status-color, item count, total, placed-at relative time, expected-delivery when present). Empty-state copy for both tabs.
- t-profile-1 -> f-profile-tab · developer · 3h - Implement GET /api/profile (returns the current retailer plus the outstanding-payments breakdown grouped by distributor) and build the Profile tab page at app/dashboard/profile/page.tsx that renders the retailer card (avatar initials, name, store name, license number, owner name, GSTIN, store address, phone, email, favourites count) and an outstanding-payments section listing each distributor's amount-due with the grand total. A logout button at the bottom POSTs /api/auth/logout and routes to /.

FULL ENGINEERING MANAGER OUTPUT (JSON)


```
{
  "features": [
    {
      "id": "f-shell-auth",
      "name": "App shell + mock retailer auth",
      "description": "Next.js 14 + Tailwind project skeleton, the login page, and the
authenticated /dashboard layout with the bottom tab bar that every other feature lives inside.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-data-spine",
      "name": "In-memory data spine + shared types",
      "description": "Single source of truth for retailers, distributors, medicines, offers, cart
items, orders, order items, and outstanding payments. In-memory Maps with seed data, plus the
shared TypeScript types every Route Handler and page imports.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-home-tab",
      "name": "Home tab",
      "description": "Search bar at the top, offers carousel, the 5 quick-link tiles
(Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments), and the
supporting catalogue / offers / distributors APIs.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-orders-tab",
      "name": "Orders tab + place-order API",
      "description": "Orders list page with Active and Closed sub-tabs, plus the POST /api/orders
fanout (cart -> one order per distributor) and GET /api/orders status-grouped reader.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-cart-tab",
      "name": "Cart tab + cart APIs",
      "description": "Cart page grouped by distributor with per-distributor subtotal and a grand
total, plus the GET / POST / DELETE cart endpoints that drive it.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-profile-tab",
      "name": "Profile tab",
```

```
      "description": "Read-only retailer card (name, owner, license number, store address, GSTIN, phone, email, favourites count) plus the GET /api/profile endpoint that returns the retailer plus the outstanding-payments breakdown.",
      "priority": "medium",
      "status": "pending"
    }
  ],
  "tasks": [
    {
      "id": "t-data-1",
      "featureId": "f-data-spine",
      "description": "Implement the in-memory data spine: lib/types.ts (TypeScript types for Retailer, Distributor, Medicine, Offer, CartItem, Order, OrderItem, OutstandingPayment, OrderStatus, derived Money helpers) and lib/db/store.ts (Map-backed stores seeded at module import with 1 pilot retailer, 4 distributors, 20 medicines spanning the four distributors, 3 active offers, 2 outstanding-payment rows; CRUD helpers - getRetailer, listDistributors, listOffers, searchMedicines(q), getMedicine(id), getCart(retailerId), addCartItem, removeCartItem, clearCart, listOrders(retailerId, statusGroup), createOrdersFromCart(retailerId), getOutstandingForRetailer(retailerId)).",
      "assignedTo": "developer",
      "estimatedHours": 4,
      "status": "pending"
    },
    {
      "id": "t-shell-1",
      "featureId": "f-shell-auth",
      "description": "Set up the Next.js 14 + Tailwind project skeleton: package.json (next 14.2, react 18, tailwind 3, typescript 5, vitest), tsconfig.json with the @/* alias, next.config.mjs, postcss.config.mjs, tailwind.config.ts (mobile-first content globs), app/globals.css with the Tailwind directives + base background, app/layout.tsx with the html/body shell, and the public login page at app/page.tsx that posts to /api/auth/login. No auth handler yet (lands in t-shell-2); the page just calls fetch and redirects on success.",
      "assignedTo": "developer",
      "estimatedHours": 4,
      "status": "pending"
    },
    {
      "id": "t-shell-2",
      "featureId": "f-shell-auth",
      "description": "Implement the mock auth: POST /api/auth/login (look up the retailer by license number against the in-memory store, set the pharmacy-session cookie HttpOnly+SameSite=Lax with a 30-day max-age), POST /api/auth/logout (clear the cookie), middleware.ts that gates /dashboard/* and every /api/* route except /api/auth/login (returns 401 JSON if missing cookie), a small lib/auth/session.ts helper that reads the current retailerId from the request cookies, and the authenticated app/dashboard/layout.tsx that renders the 4-tab bottom navigation (Home / Orders / Cart / Profile) and a stub app/dashboard/page.tsx that redirects to /dashboard/home.",
      "assignedTo": "developer",
      "estimatedHours": 4,
      "status": "pending"
    },
    {
      "id": "t-home-1",
      "featureId": "f-home-tab",
      "description": "Implement the read APIs that drive the Home tab: GET /api/medicines/search (case-insensitive substring over name + brand + generic, returns each result with its distributor), GET /api/distributors (id/name/region/rating list), GET /api/offers (active offers in sortOrder, each with its distributor). All three use the auth-guarded session helper to identify the retailer (search results may carry per-retailer favourites later) and return 401 if the cookie is missing.",
      "assignedTo": "developer",
      "estimatedHours": 3,
      "status": "pending"
    },
    {
      "id": "t-home-2",
      "featureId": "f-home-tab",
      "description": "Build the Home tab page at app/dashboard/home/page.tsx: greeting line with the retailer name, a search bar at the top that links to /dashboard/search?q=..., a horizontally scrollable offers carousel reading from GET /api/offers, the 5 quick-link tiles (Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments), and an outstanding-payments summary tile that calls GET /api/profile and shows the total + per-distributor breakdown. Mobile-first Tailwind layout, snap-scroll on the carousel.",
      "assignedTo": "developer",
      "estimatedHours": 4,
      "status": "pending"
```

```
    },
    {
      "id": "t-cart-1",
      "featureId": "f-cart-tab",
      "description": "Implement the cart APIs: GET /api/cart (returns the cart grouped by distributor with subtotals + grand total + itemCount, derived from the in-memory store), POST /api/cart/items (validates medicineId exists, increments qty if already present, returns the same grouped payload), DELETE /api/cart/items/[medicineId] (removes the line, returns the same grouped payload). All three resolve the retailer via the session helper and 401 without it.",
      "assignedTo": "developer",
      "estimatedHours": 3,
      "status": "pending"
    },
    {
      "id": "t-cart-2",
      "featureId": "f-cart-tab",
      "description": "Build the Cart tab page at app/dashboard/cart/page.tsx: groups the cart by distributor (heading per supplier with subtotal), each line shows name + brand + qty stepper + remove button + line total, sticky footer with the grand total and a Place Order button that POSTs to /api/orders and on success routes to /dashboard/orders?status=active. Empty-state copy when the cart is empty.",
      "assignedTo": "developer",
      "estimatedHours": 3,
      "status": "pending"
    },
    {
      "id": "t-orders-1",
      "featureId": "f-orders-tab",
      "description": "Implement the orders APIs: GET /api/orders?status=active|closed (active = placed | acknowledged | out_for_delivery, closed = delivered | cancelled, sorted newest-first), POST /api/orders (reads the current retailer's cart, fans out into one order per distinct distributorId, copies cart_items into order_items, clears the cart, returns the new orderIds + count). 401 without the session cookie.",
      "assignedTo": "developer",
      "estimatedHours": 3,
      "status": "pending"
    },
    {
      "id": "t-orders-2",
      "featureId": "f-orders-tab",
      "description": "Build the Orders tab page at app/dashboard/orders/page.tsx with Active and Closed sub-tabs (segmented control). Each sub-tab calls GET /api/orders with the matching status group and renders order cards (distributor name, status pill with status-color, item count, total, placed-at relative time, expected-delivery when present). Empty-state copy for both tabs.",
      "assignedTo": "developer",
      "estimatedHours": 3,
      "status": "pending"
    },
    {
      "id": "t-profile-1",
      "featureId": "f-profile-tab",
      "description": "Implement GET /api/profile (returns the current retailer plus the outstanding-payments breakdown grouped by distributor) and build the Profile tab page at app/dashboard/profile/page.tsx that renders the retailer card (avatar initials, name, store name, license number, owner name, GSTIN, store address, phone, email, favourites count) and an outstanding-payments section listing each distributor's amount-due with the grand total. A logout button at the bottom POSTs /api/auth/logout and routes to /.",
      "assignedTo": "developer",
      "estimatedHours": 3,
      "status": "pending"
    }
  ]
}
```

Developer

Task: t-data-1

IMPLEMENTATION PLAN

Build the in-memory data spine the rest of the app reads from. lib/types.ts ships every shared TypeScript type (Retailer, Distributor, Medicine, Offer, CartItem, Order, OrderItem, OutstandingPayment, OrderStatus, OrderStatusGroup), the canonical ACTIVE_STATUSES / CLOSED_STATUSES sets, the STATUS_LABELS map, isActiveStatus / statusGroup helpers, and a paiseToRupees(amount) helper using Indian numbering (1,23,45,678) with a Rs symbol. lib/db/store.ts holds Map-backed stores keyed by id (or composite for cart_items / outstanding) plus the public API every Route Handler needs: getRetailer, findRetailerByLicense, listDistributors, getDistributor, listOffers (filters expired), searchMedicines (case-insensitive over name + brand + generic), getMedicine, listCartItems, addCartItem (idempotent on (retailerId, medicineId) - increments qty), removeCartItem, clearCart, listOrders (grouped by status), getOrder, listOrderItems, createOrdersFromCart (fan-out one Order per distinct distributorId in the cart, copy CartItem rows into OrderItem rows, clearCart on success), getOutstandingForRetailer. Seed data lands behind a Symbol.for('pharmacy-b2b.store.seeded') flag on globalThis so Next's hot module reload doesn't re-seed and double rows. Seed includes 4 distributors, 20 medicines (5 per distributor), 3 active offers, the pilot retailer (license MH-RP-2024-7821 to match t-shell-1's default), 2 outstanding-payment rows, and 2 pre-existing orders (one out_for_delivery, one delivered) so both Orders sub-tabs render content on first load. Money is stored as integer paise everywhere; conversion to display string is the formatter's job.

FILES PRODUCED (2)

- `lib/db/store.ts (15,207 bytes)`
- `lib/types.ts (3,031 bytes)`

TESTS (4)

- lib/types.ts: ACTIVE_STATUSES, CLOSED_STATUSES, STATUS_LABELS cover every OrderStatus exactly once.
- lib/types.ts paiseToRupees uses Indian numbering and 2-decimal padding.
- lib/db/store.ts: seed data populates retailers / distributors / medicines / offers / outstanding / orders.
- lib/db/store.ts: cart and order fan-out invariants.

NOTES (4)

- Money is stored as integer paise everywhere; the only formatting helper (paiseToRupees) lives in lib/types.ts so it can be imported from both Route Handlers (server) and pages (client/server). Two-decimal padding + Indian numbering match the audience's expectation.
- All Map instances live on globalThis under a single namespaced key (__pharmacyB2BStore). Next.js dev mode loads each Route Handler in its own module graph, which means a plain `const x = new Map()` at module scope produces a FRESH Map per handler - the login handler and the search handler then see different stores. Hoisting onto globalThis is the canonical Next 14 pattern for shared in-memory state across all server modules in one process; without it, /api/auth/login finds the retailer but /api/medicines/search returns []. The seeded flag also lives inside the globalThis-backed object so Next's hot reload + Route Handler re-imports do not produce duplicate rows.
- The pilot retailer license number 'MH-RP-2024-7821' deliberately matches the default value pre-filled by the t-shell-1 login page so a fresh `npm run dev` works end-to-end on the first try.
- createOrdersFromCart fans out one Order per distinct distributorId, copies CartItem rows into OrderItem rows with frozen unit prices, and clears the cart atomically (within the synchronous block) - mirrors the contract POST /api/orders is documented to honour.

FULL DEVELOPER OUTPUT (JSON)


```
{
  "taskId": "t-data-1",
  "implementationPlan": "Build the in-memory data spine the rest of the app reads from. lib/types.ts ships every shared TypeScript type (Retailer, Distributor, Medicine, Offer, CartItem, Order, OrderItem, OutstandingPayment, OrderStatus, OrderStatusGroup), the canonical ACTIVE_STATUSES / CLOSED_STATUSES sets, the STATUS_LABELS map, isActiveStatus / statusGroup helpers, and a paiseToRupees(amount) helper using Indian numbering (1,23,45,678) with a Rs symbol. lib/db/store.ts holds Map-backed stores keyed by id (or composite for cart_items / outstanding) plus the public API every Route Handler needs: getRetailer, findRetailerByLicense, listDistributors, getDistributor, listOffers (filters expired), searchMedicines (case-insensitive over name + brand + generic), getMedicine, listCartItems, addCartItem (idempotent on (retailerId, medicineId) - increments qty), removeCartItem, clearCart, listOrders (grouped by status), getOrder, listOrderItems, createOrdersFromCart (fan-out one Order per distinct distributorId in the cart, copy CartItem rows into OrderItem rows, clearCart on success), getOutstandingForRetailer. Seed data lands behind a Symbol.for('pharmacy-b2b.store.seeded') flag on globalThis so Next's hot module reload doesn't re-seed and double rows. Seed includes 4 distributors, 20 medicines (5 per distributor), 3 active offers, the pilot retailer (license MH-RP-2024-7821 to match t-shell-1's default), 2 outstanding-payment rows, and 2 pre-existing orders (one out_for_delivery, one delivered) so both Orders sub-tabs render content on first load. Money is stored as integer paise everywhere; conversion to display string is the formatter's job.",
  "files": [
    {
      "path": "lib/db/store.ts",
      "content": "import { randomUUID } from \"node:crypto\";\n\nimport {\n  ACTIVE_STATUSES,\n  CLOSED_STATUSES,\n  type CartItem,\n  type Distributor,\n  type Medicine,\n  type Offer,\n  type Order,\n  type OrderItem,\n  type OrderStatusGroup,\n  type OutstandingPayment,\n  type Retailer,\n} from \"@/lib/types\";\n\n// ---------- in-memory tables ----------\n//\n// IMPORTANT: the Map instances live on globalThis under a single namespaced\n// key. Next.js dev mode loads each Route Handler in its own module graph,\n// which means a plain `const x = new Map()` at module scope produces a\n// FRESH Map per handler - the login handler and the search handler then see\n// different stores. Hoisting onto globalThis is the canonical Next 14\n// pattern for \"shared in-memory state across all server modules in one\n// process\". See https://nextjs.org/docs/app/building-your-application/data-fetching\n\ninterface PharmacyStore {\n  retailers: Map<string, Retailer>;\n  distributors: Map<string, Distributor>;\n  medicines: Map<string, Medicine>;\n  offers: Map<string, Offer>;\n  cartItems: Map<string, CartItem>;\n  orders: Map<string, Order>;\n  orderItems: Map<string, OrderItem[]>;\n  outstanding: Map<string, OutstandingPayment>;\n  seeded: boolean;\n}\n\nconst STORE_KEY = \"__pharmacyB2BStore\";\ntype GlobalWithStore = typeof globalThis & { [STORE_KEY]?: PharmacyStore };\n\nfunction getStore(): PharmacyStore {\n  const g = globalThis as GlobalWithStore;\n  let s = g[STORE_KEY];\n  if (!s) {\n    s = {\n      retailers: new Map(),\n      distributors: new Map(),\n      medicines: new Map(),\n      offers: new Map(),\n      cartItems: new Map(),\n      orders: new Map(),\n      orderItems: new Map(),\n      outstanding: new Map(),\n      seeded: false,\n    };\n    g[STORE_KEY] = s;\n  }\n  if (!s.seeded) {\n    seed(s);\n    s.seeded = true;\n  }\n  return s;\n}\n\nconst STORE = getStore();\nconst retailers = STORE.retailers;\nconst distributors = STORE.distributors;\nconst medicines = STORE.medicines;\nconst offers = STORE.offers;\nconst cartItems = STORE.cartItems; // key = `${retailerId}|${medicineId}`\nconst orders = STORE.orders;\nconst orderItems = STORE.orderItems; // key = orderId\nconst outstanding = STORE.outstanding; // key = `${retailerId}|${distributorId}`\n\n// ---------- public API ----------\n\nexport function getRetailer(retailerId: string): Retailer | undefined {\n  return retailers.get(retailerId);\n}\n\nexport function findRetailerByLicense(licenseNumber: string): Retailer | undefined {\n  for (const r of retailers.values()) {\n    if (r.licenseNumber.toLowerCase() === licenseNumber.toLowerCase()) return r;\n  }\n  return undefined;\n}\n\nexport function listDistributors(): Distributor[] {\n  return Array.from(distributors.values()).sort((a, b) => a.name.localeCompare(b.name));\n}\n\nexport function getDistributor(id: string): Distributor | undefined {\n  return distributors.get(id);\n}\n\nexport function listOffers(): Offer[] {\n  const now = Date.now();\n  return Array.from(offers.values())\n    .filter((o) => o.validUntil.getTime() >= now)\n    .sort((a, b) => a.sortOrder - b.sortOrder);\n}\n\nexport function searchMedicines(query: string): Medicine[] {\n  const q = query.trim().toLowerCase();\n  const all = Array.from(medicines.values());\n  if (!q) return all.sort((a, b) => a.name.localeCompare(b.name));\n  return all\n    .filter(\n      (m) =>\n        m.name.toLowerCase().includes(q) ||\n        m.brand.toLowerCase().includes(q) ||\n        m.genericName.toLowerCase().includes(q),\n    )\n    .sort((a, b) => a.name.localeCompare(b.name));\n}\n\nexport function getMedicine(id: string): Medicine | undefined {\n  return medicines.get(id);\n}\n\nexport function listCartItems(retailerId: string): CartItem[] {\n  return Array.from(cartItems.values())\n    .filter((c) => c.retailerId === retailerId)\n    .sort((a, b) => a.addedAt.getTime() - b.addedAt.getTime());\n}\n\nexport function addCartItem(retailerId: string, medicineId: string, qty: number): CartItem {\n  if (qty < 1) throw new Error(\"addCartItem: qty must be >= 1\");\n  const med = medicines.get(medicineId);\n  if (!med) throw new Error(`addCartItem: unknown medicineId ${medicineId}`);\n  const key = `${retailerId}|${medicineId}`;\n  const existing = cartItems.get(key);\n  if (existing) {\n    existing.qty += qty;\n    return existing;\n  }\n  const item: CartItem = {\n    id: randomUUID(),\n    retailerId,\n    medicineId,\n    distributorId: med.distributorId,\n    qty,\n    unitPricePaise: med.sellingPricePaise,\n    addedAt: new Date(),\n  };\n  cartItems.set(key,
```

```
item);\n  return item;\n}\n\nexport function removeCartItem(retailerId: string, medicineId:
string): boolean {\n  const key = `${retailerId}|${medicineId}`;\n  return cartItems.delete(key);
\n}\n\nexport function clearCart(retailerId: string): number {\n  let removed = 0;\n  for (const
[k, v] of cartItems.entries()) {\n    if (v.retailerId === retailerId) {\n
cartItems.delete(k);\n      removed++;\n    }\n  }\n  return removed;\n}\n\nexport function
listOrders(retailerId: string, group: OrderStatusGroup): Order[] {\n  const set = group ===
\"active\" ? ACTIVE_STATUSES : CLOSED_STATUSES;\n  return
Array.from(orders.values())\n    .filter((o) => o.retailerId === retailerId &&
set.has(o.status))\n    .sort((a, b) => b.placedAt.getTime() - a.placedAt.getTime());
\n}\n\nexport function getOrder(id: string): Order | undefined {\n  return orders.get(id);
\n}\n\nexport function listOrderItems(orderId: string): OrderItem[] {\n  return
orderItems.get(orderId) ?? [];\n}\n\nexport interface CreatedOrder {\n  id: string;\n
distributorId: string;\n}\n\nexport function createOrdersFromCart(retailerId: string):
CreatedOrder[] {\n  const items = listCartItems(retailerId);\n  if (items.length === 0) return [];
\n\n  const groups = new Map<string, CartItem[]>();\n  for (const item of items) {\n    const
list = groups.get(item.distributorId) ?? [];\n    list.push(item);\n
groups.set(item.distributorId, list);\n  }\n\n  const created: CreatedOrder[] = [];\n  for (const
[distributorId, lines] of groups) {\n    const orderId = randomUUID();\n    const total =
lines.reduce((acc, l) => acc + l.qty * l.unitPricePaise, 0);\n    const itemCount =
lines.reduce((acc, l) => acc + l.qty, 0);\n    const order: Order = {\n      id: orderId,\n
retailerId,\n      distributorId,\n      status: \"placed\",\n      placedAt: new Date(),\n
expectedDelivery: addDays(new Date(), 2),\n      totalPaise: total,\n      itemCount,\n    };
\n    orders.set(orderId, order);\n    orderItems.set(\n      orderId,\n
lines.map<OrderItem>((l) => ({\n        id: randomUUID(),\n        orderId,\n        medicineId:
l.medicineId,\n        qty: l.qty,\n        unitPricePaise: l.unitPricePaise,\n
lineTotalPaise: l.qty * l.unitPricePaise,\n      })),\n    );\n    created.push({ id: orderId,
distributorId });\n  }\n\n  clearCart(retailerId);\n  return created;\n}\n\nexport function
getOutstandingForRetailer(retailerId: string): OutstandingPayment[] {\n  return
Array.from(outstanding.values())\n    .filter((p) => p.retailerId === retailerId)\n    .sort((a,
b) => b.amountDuePaise - a.amountDuePaise);\n}\n\n// ---------- seed ----------\n\nfunction
seed(s: PharmacyStore): void {\n  const now = new Date();\n\n  // Distributors\n  const
dMediplus: Distributor = mkDist(\"d-mediplus\", \"MediPlus Distributors\", \"Mumbai\",
\"MEDIPLUS-001\", \"+91 22 4001 8821\", \"orders@mediplus.in\", 4.6, now);\n  const dOmsai:
Distributor = mkDist(\"d-omsai\", \"Om Sai Pharma\", \"Pune\", \"OMSAI-002\", \"+91 20 6711
4422\", \"ops@omsaipharma.in\", 4.4, now);\n  const dAakash: Distributor = mkDist(\"d-aakash\",
\"Aakash Wholesale\", \"Nashik\", \"AAKASH-003\", \"+91 253 2580 9991\",
\"trade@aakashwholesale.in\", 4.2, now);\n  const dPharma: Distributor = mkDist(\"d-pharmacare\",
\"PharmaCare Hub\", \"Mumbai\", \"PCARE-004\", \"+91 22 4900 7711\", \"hello@pharmacarehub.in\",
4.7, now);\n  for (const d of [dMediplus, dOmsai, dAakash, dPharma]) s.distributors.set(d.id, d);
\n\n  // Medicines (5 per distributor)\n  const meds: Medicine[] = [\n    mkMed(\"m-
paracetamol-500\", \"Paracetamol 500mg\", \"Crocin\", \"GSK\", \"Acetaminophen\", \"d-mediplus\",
25, 22, \"10+1 free\", \"10 tablets\", \"30049099\", now),\n    mkMed(\"m-amoxicillin-500\",
\"Amoxicillin 500mg cap\", \"Mox\", \"Sun Pharma\", \"Amoxicillin\", \"d-mediplus\", 95, 84,
null, \"10 capsules\", \"30041000\", now),\n    mkMed(\"m-pantoprazole-40\", \"Pantoprazole
40mg\", \"Pantop\", \"Aristo\", \"Pantoprazole\", \"d-mediplus\", 110, 98, null, \"15 tablets\",
\"30049099\", now),\n    mkMed(\"m-azithromycin-500\", \"Azithromycin 500mg\", \"Azithral\",
\"Alembic\", \"Azithromycin\", \"d-mediplus\", 145, 130, null, \"5 tablets\", \"30042090\", now),
\n    mkMed(\"m-cetirizine-10\", \"Cetirizine 10mg\", \"Cetzine\", \"GSK\", \"Cetirizine\", \"d-
mediplus\", 22, 19, \"20% off\", \"10 tablets\", \"30049099\", now),\n\n    mkMed(\"m-
vitd3-60k\", \"Vitamin D3 60K IU\", \"Calcirol\", \"Cadila\", \"Cholecalciferol\", \"d-omsai\",
35, 31, null, \"4 sachets\", \"30049030\", now),\n    mkMed(\"m-iron-folic\", \"Iron + Folic
acid\", \"Livogen\", \"Merck\", \"Iron Folic Acid\", \"d-omsai\", 65, 58, null, \"30 tablets\",
\"30049036\", now),\n    mkMed(\"m-omeprazole-20\", \"Omeprazole 20mg\", \"Omez\", \"Dr Reddys\",
\"Omeprazole\", \"d-omsai\", 78, 70, \"Buy 2 get 1 free\", \"10 capsules\", \"30049099\", now),
\n    mkMed(\"m-aspirin-150\", \"Aspirin 150mg\", \"Ecosprin\", \"USV\", \"Aspirin\", \"d-
omsai\", 45, 39, null, \"14 tablets\", \"30049099\", now),\n    mkMed(\"m-metformin-500\",
\"Metformin 500mg\", \"Glycomet\", \"USV\", \"Metformin\", \"d-omsai\", 55, 48, null, \"20
tablets\", \"30049099\", now),\n\n    mkMed(\"m-amlodipine-5\", \"Amlodipine 5mg\", \"Amlong\",
\"Micro Labs\", \"Amlodipine\", \"d-aakash\", 38, 33, null, \"10 tablets\", \"30049099\", now),
\n    mkMed(\"m-telmisartan-40\", \"Telmisartan 40mg\", \"Telma\", \"Glenmark\", \"Telmisartan\",
\"d-aakash\", 92, 81, null, \"15 tablets\", \"30049099\", now),\n    mkMed(\"m-losartan-50\",
\"Losartan 50mg\", \"Losar\", \"Unichem\", \"Losartan\", \"d-aakash\", 67, 60, \"10+2 free\",
\"10 tablets\", \"30049099\", now),\n    mkMed(\"m-atorvastatin-10\", \"Atorvastatin 10mg\",
\"Atorlip\", \"Cipla\", \"Atorvastatin\", \"d-aakash\", 89, 78, null, \"10 tablets\",
\"30049099\", now),\n    mkMed(\"m-rosuvastatin-10\", \"Rosuvastatin 10mg\", \"Rosuvas\", \"Sun
Pharma\", \"Rosuvastatin\", \"d-aakash\", 134, 119, null, \"10 tablets\", \"30049099\", now),
\n\n    mkMed(\"m-insulin-pen\", \"Insulin Glargine 100IU pen\", \"Lantus\", \"Sanofi\",
\"Insulin Glargine\", \"d-pharmacare\", 1850, 1620, null, \"1 pen\", \"30043190\", now),\n
mkMed(\"m-metformin-er-1k\", \"Metformin ER 1000mg\", \"Glycomet GP\", \"USV\", \"Metformin\",
\"d-pharmacare\", 124, 109, null, \"15 tablets\", \"30049099\", now),\n    mkMed(\"m-
thyroxine-50\", \"Thyroxine 50mcg\", \"Thyronorm\", \"Abbott\", \"Levothyroxine\", \"d-
pharmacare\", 145, 128, null, \"100 tablets\", \"30049099\", now),\n    mkMed(\"m-
clopidogrel-75\", \"Clopidogrel 75mg\", \"Plavix\", \"Sanofi\", \"Clopidogrel\", \"d-
pharmacare\", 168, 148, null, \"10 tablets\", \"30049099\", now),\n    mkMed(\"m-
ranitidine-150\", \"Ranitidine 150mg\", \"Rantac\", \"JBChem\", \"Ranitidine\", \"d-pharmacare\",
```

```
28, 24, \"30% off\", \"10 tablets\", \"30049099\", now),\n  ];\n  for (const m of meds) s.medicines.set(m.id, m);\n\n  // Offers\n  s.offers.set(\"o-1\", {\n    id: \"o-1\",\n    title: \"Save 10% on antibiotics\",\n    description: \"Flat 10% off on Mox, Azithral and other antibiotics from MediPlus.\",\n    distributorId: \"d-mediplus\",\n    bannerLabel: \"10% OFF\",\n    validUntil: addDays(now, 28),\n    sortOrder: 1,\n    createdAt: now,\n  });\n  s.offers.set(\"o-2\", {\n    id: \"o-2\",\n    title: \"Free delivery on orders over Rs.5,000\",\n    description: \"Om Sai Pharma waives the delivery fee on every order above Rs.5,000.\",\n    distributorId: \"d-omsai\",\n    bannerLabel: \"FREE SHIPPING\",\n    validUntil: addDays(now, 14),\n    sortOrder: 2,\n    createdAt: now,\n  });\n  s.offers.set(\"o-3\", {\n    id: \"o-3\",\n    title: \"Bulk diabetic care - extra 5% off\",\n    description: \"Order Insulin pens, Metformin or Glycomet GP from PharmaCare Hub and get an extra 5% off above 10 units.\",\n    distributorId: \"d-pharmacare\",\n    bannerLabel: \"DIABETIC CARE\",\n    validUntil: addDays(now, 21),\n    sortOrder: 3,\n    createdAt: now,\n  });\n\n  // Pilot retailer\n  const pilot: Retailer = {\n    id: \"r-pilot-1\",\n    name: \"Anuradha Medicals\",\n    ownerName: \"Suresh Kulkarni\",\n    licenseNumber: \"MH-RP-2024-7821\",\n    storeName: \"Anuradha Medicals\",\n    storeAddress: \"Plot 14, Shivajinagar, Pune 411005\",\n    phone: \"+91 90220 14857\",\n    email: \"suresh@anuradhamedicals.in\",\n    gstin: \"27ABCDE1234F1Z5\",\n    favouritesMedicineIds: [\"m-paracetamol-500\", \"m-pantoprazole-40\", \"m-vitd3-60k\"],\n    createdAt: now,\n  };\n  s.retailers.set(pilot.id, pilot);\n\n  // Outstanding payments\n  s.outstanding.set(`${pilot.id}|d-mediplus`, {\n    id: \"op-1\",\n    retailerId: pilot.id,\n    distributorId: \"d-mediplus\",\n    amountDuePaise: 1_425_000,\n    lastUpdatedAt: addDays(now, -2),\n  });\n  s.outstanding.set(`${pilot.id}|d-pharmacare`, {\n    id: \"op-2\",\n    retailerId: pilot.id,\n    distributorId: \"d-pharmacare\",\n    amountDuePaise: 875_000,\n    lastUpdatedAt: addDays(now, -5),\n  });\n\n  // Pre-existing orders so the Orders tab is non-empty on first load.\n  seedOrder(\n    s,\n    \"ord-seed-1\",\n    pilot.id,\n    \"d-omsai\",\n    \"out_for_delivery\",\n    addDays(now, -1),\n    addDays(now, 1),\n    [\n      { medicineId: \"m-iron-folic\", qty: 3, unitPricePaise: 5_800 },\n      { medicineId: \"m-omeprazole-20\", qty: 5, unitPricePaise: 7_000 },\n    ],\n  );\n  seedOrder(\n    s,\n    \"ord-seed-2\",\n    pilot.id,\n    \"d-aakash\",\n    \"delivered\",\n    addDays(now, -8),\n    addDays(now, -6),\n    [\n      { medicineId: \"m-amlodipine-5\", qty: 10, unitPricePaise: 3_300 },\n      { medicineId: \"m-telmisartan-40\", qty: 4, unitPricePaise: 8_100 },\n    ],\n  );\n}\n\ninterface SeedLine {\n  medicineId: string;\n  qty: number;\n  unitPricePaise: number;\n}\n\nfunction seedOrder(\n  s: PharmacyStore,\n  id: string,\n  retailerId: string,\n  distributorId: string,\n  status: Order[\"status\"],\n  placedAt: Date,\n  expectedDelivery: Date | null,\n  lines: SeedLine[],\n): void {\n  const total = lines.reduce((acc, l) => acc + l.qty * l.unitPricePaise, 0);\n  const itemCount = lines.reduce((acc, l) => acc + l.qty, 0);\n  s.orders.set(id, {\n    id,\n    retailerId,\n    distributorId,\n    status,\n    placedAt,\n    expectedDelivery,\n    totalPaise: total,\n    itemCount,\n  });\n  s.orderItems.set(\n    id,\n    lines.map((l, i) => ({\n      id: `${id}-i-${i}`,\n      orderId: id,\n      medicineId: l.medicineId,\n      qty: l.qty,\n      unitPricePaise: l.unitPricePaise,\n      lineTotalPaise: l.qty * l.unitPricePaise,\n    })),\n  );\n}\n\nfunction mkDist(\n  id: string,\n  name: string,\n  region: string,\n  supplierCode: string,\n  contactPhone: string,\n  contactEmail: string,\n  rating: number,\n  createdAt: Date,\n): Distributor {\n  return { id, name, region, supplierCode, contactPhone, contactEmail, rating, createdAt };\n}\n\nfunction mkMed(\n  id: string,\n  name: string,\n  brand: string,\n  manufacturer: string,\n  genericName: string,\n  distributorId: string,\n  mrpRupees: number,\n  sellingRupees: number,\n  scheme: string | null,\n  packSize: string,\n  hsnCode: string,\n  createdAt: Date,\n): Medicine {\n  return {\n    id,\n    name,\n    brand,\n    manufacturer,\n    genericName,\n    distributorId,\n    mrpPaise: mrpRupees * 100,\n    sellingPricePaise: sellingRupees * 100,\n    scheme,\n    packSize,\n    hsnCode,\n    createdAt,\n  };\n}\n\nfunction addDays(d: Date, n: number): Date {\n  const out = new Date(d);\n  out.setDate(out.getDate() + n);\n  return out;\n}\n"
    },
    {
      "path": "lib/types.ts",
      "content": "export type OrderStatus =\n  | \"placed\"\n  | \"acknowledged\"\n  | \"out_for_delivery\"\n  | \"delivered\"\n  | \"cancelled\";\n\nexport type OrderStatusGroup = \"active\" | \"closed\";\n\nexport const ACTIVE_STATUSES: ReadonlySet<OrderStatus> = new Set([\n  \"placed\",\n  \"acknowledged\",\n  \"out_for_delivery\",\n]);\n\nexport const CLOSED_STATUSES: ReadonlySet<OrderStatus> = new Set([\n  \"delivered\",\n  \"cancelled\",\n]);\n\nexport const STATUS_LABELS: Record<OrderStatus, string> = {\n  placed: \"Placed\",\n  acknowledged: \"Acknowledged\",\n  out_for_delivery: \"Out for delivery\",\n  delivered: \"Delivered\",\n  cancelled: \"Cancelled\",\n};\n\nexport interface Retailer {\n  id: string;\n  name: string;\n  ownerName: string;\n  licenseNumber: string;\n  storeName: string;\n  storeAddress: string;\n  phone: string;\n  email: string;\n  gstin: string;\n  favouritesMedicineIds: string[];\n  createdAt: Date;\n}\n\nexport interface Distributor {\n  id: string;\n  name: string;\n  region: string;\n  supplierCode: string;\n  contactPhone: string;\n  contactEmail: string;\n  rating: number;\n  createdAt: Date;\n}\n\nexport interface Medicine {\n  id: string;\n  name: string;\n  brand: string;\n  manufacturer: string;\n  genericName: string;\n  distributorId: string;\n  mrpPaise: number;\n  sellingPricePaise: number;\n  scheme: string | null;\n  packSize: string;\n  hsnCode: string;\n  createdAt: Date;\n}\n\nexport interface Offer {\n  id: string;\n  title: string;\n  description: string;\n  distributorId: string;\n  bannerLabel: string;\n  validUntil: Date;\n  sortOrder: number;\n  createdAt: Date;\n}\n\nexport interface CartItem {\n  id: string;\n  retailerId: string;\n  medicineId: string;\n  distributorId: string;\n  qty: number;\n  unitPricePaise: number;\n  addedAt: Date;\n}\n\nexport interface Order {\n  id: string;\n  retailerId: string;\n  distributorId: string;\n  status: OrderStatus;\n  placedAt: Date;\n
```

```
expectedDelivery: Date | null;\n  totalPaise: number;\n  itemCount: number;\n}\n\nexport
interface OrderItem {\n  id: string;\n  orderId: string;\n  medicineId: string;\n  qty: number;
\n  unitPricePaise: number;\n  lineTotalPaise: number;\n}\n\nexport interface OutstandingPayment
{\n  id: string;\n  retailerId: string;\n  distributorId: string;\n  amountDuePaise: number;\n
lastUpdatedAt: Date;\n}\n\nexport function isActiveStatus(status: OrderStatus): boolean {\n
return ACTIVE_STATUSES.has(status);\n}\n\nexport function statusGroup(status: OrderStatus):
OrderStatusGroup {\n  return ACTIVE_STATUSES.has(status) ? \"active\" : \"closed\";\n}\n\nexport
function paiseToRupees(paise: number): string {\n  const sign = paise < 0 ? \"-\" : \"\";\n
const abs = Math.abs(paise);\n  const rupees = Math.floor(abs / 100);\n  const paisePart = (abs %
100).toString().padStart(2, \"0\");\n  return `${sign}\\u20B9${formatThousands(rupees)}.
${paisePart}`;\n}\n\nfunction formatThousands(n: number): string {\n  // Indian numbering system:
1,23,45,678\n  const s = String(n);\n  if (s.length <= 3) return s;\n  const last3 = s.slice(-3);
\n  const rest = s.slice(0, -3);\n  const grouped = rest.replace(/\\B(?=(\\d{2})+(?!\\d))/g, \",
\");\n  return `${grouped},${last3}`;\n}\n"
    }
  ],
  "tests": [
    {
      "description": "lib/types.ts: ACTIVE_STATUSES, CLOSED_STATUSES, STATUS_LABELS cover every
OrderStatus exactly once.",
      "code": "import { describe, expect, it } from \"vitest\";\nimport { ACTIVE_STATUSES,
CLOSED_STATUSES, STATUS_LABELS, type OrderStatus, isActiveStatus, statusGroup } from \"./types\";
\n\nconst ALL: OrderStatus[] = [\"placed\", \"acknowledged\", \"out_for_delivery\",
\"delivered\", \"cancelled\"];\n\ndescribe(\"order status taxonomy\", () => {\n  it(\"every
status is in exactly one of ACTIVE_STATUSES / CLOSED_STATUSES\", () => {\n    for (const s of
ALL) {\n      const inActive = ACTIVE_STATUSES.has(s);\n      const inClosed =
CLOSED_STATUSES.has(s);\n      expect(inActive !== inClosed).toBe(true);\n    }\n  });\n\n
it(\"every status has a STATUS_LABELS entry\", () => {\n    for (const s of ALL)
expect(STATUS_LABELS[s]).toBeTruthy();\n  });\n\n  it(\"isActiveStatus + statusGroup agree with
ACTIVE_STATUSES\", () => {\n    for (const s of ALL) {\n
expect(isActiveStatus(s)).toBe(ACTIVE_STATUSES.has(s));\n
expect(statusGroup(s)).toBe(ACTIVE_STATUSES.has(s) ? \"active\" : \"closed\");\n    }\n  });\n});
\n"
    },
    {
      "description": "lib/types.ts paiseToRupees uses Indian numbering and 2-decimal padding.",
      "code": "import { describe, expect, it } from \"vitest\";\nimport { paiseToRupees } from
\"./types\";\n\ndescribe(\"paiseToRupees\", () => {\n  it(\"renders zero as Rs.0.00\", () =>
{\n    expect(paiseToRupees(0)).toBe(\"\\u20B90.00\");\n  });\n  it(\"renders 1234567 paise as
Rs.12,345.67 (Indian grouping)\", () => {\n    expect(paiseToRupees(1234567)).toBe(\"\
\u20B912,345.67\");\n  });\n  it(\"renders 100000000 paise as Rs.10,00,000.00\", () => {\n
expect(paiseToRupees(100_000_000)).toBe(\"\\u20B910,00,000.00\");\n  });\n  it(\"prefixes
negatives with -\", () => {\n    expect(paiseToRupees(-150)).toBe(\"-\\u20B91.50\");\n  });\n});
\n"
    },
    {
      "description": "lib/db/store.ts: seed data populates retailers / distributors / medicines /
offers / outstanding / orders.",
      "code": "import { describe, expect, it } from \"vitest\";\nimport { findRetailerByLicense,
listDistributors, listOffers, searchMedicines, listOrders, getOutstandingForRetailer } from \"./
store\";\n\ndescribe(\"in-memory store seed\", () => {\n  const PILOT = \"r-pilot-1\";\n\n
it(\"finds the pilot retailer by license number (case-insensitive)\", () => {\n
expect(findRetailerByLicense(\"MH-RP-2024-7821\")?.id).toBe(PILOT);\n
expect(findRetailerByLicense(\"mh-rp-2024-7821\")?.id).toBe(PILOT);\n  });\n\n  it(\"lists all 4
distributors sorted by name\", () => {\n    const d = listDistributors();\n
expect(d).toHaveLength(4);\n    const names = d.map((x) => x.name);\n
expect(names).toEqual([...names].sort());\n  });\n\n  it(\"lists 3 active offers in sortOrder\",
() => {\n    const o = listOffers();\n    expect(o).toHaveLength(3);\n    expect(o.map((x) =>
x.sortOrder)).toEqual([1, 2, 3]);\n  });\n\n  it(\"empty search returns the full catalogue (20
medicines)\", () => {\n    expect(searchMedicines(\"\")).toHaveLength(20);\n  });\n\n
it(\"search 'paracet' is case-insensitive over name/brand/generic\", () => {\n    const r =
searchMedicines(\"paracet\");\n    expect(r.length).toBeGreaterThanOrEqual(1);\n
expect(r.some((m) => m.id === \"m-paracetamol-500\")).toBe(true);\n  });\n\n  it(\"seeded orders
split between active and closed groups\", () => {\n    const active = listOrders(PILOT,
\"active\");\n    const closed = listOrders(PILOT, \"closed\");\n
expect(active.length).toBeGreaterThanOrEqual(1);\n
expect(closed.length).toBeGreaterThanOrEqual(1);\n    expect(active.every((o) => o.status ===
\"placed\" || o.status === \"acknowledged\" || o.status === \"out_for_delivery\")).toBe(true);
\n    expect(closed.every((o) => o.status === \"delivered\" || o.status ===
\"cancelled\")).toBe(true);\n  });\n\n  it(\"outstanding payments are seeded for the pilot
retailer\", () => {\n    const op = getOutstandingForRetailer(PILOT);\n
expect(op.length).toBe(2);\n    expect(op.reduce((acc, x) => acc + x.amountDuePaise,
0)).toBe(2_300_000);\n  });\n});\n"
    },
    {
```

```
      "description": "lib/db/store.ts: cart and order fan-out invariants.",
      "code": "import { describe, expect, it, beforeEach } from \"vitest\";\nimport
{ addCartItem, clearCart, listCartItems, createOrdersFromCart, getOrder, listOrderItems,
removeCartItem } from \"./store\";\n\nconst PILOT = \"r-pilot-1\";\n\ndescribe(\"cart and order
fan-out\", () => {\n  beforeEach(() => clearCart(PILOT));\n\n  it(\"addCartItem is idempotent on
(retailerId, medicineId): adds qty\", () => {\n    addCartItem(PILOT, \"m-paracetamol-500\", 1);
\n    addCartItem(PILOT, \"m-paracetamol-500\", 2);\n    const items = listCartItems(PILOT);\n
expect(items).toHaveLength(1);\n    expect(items[0].qty).toBe(3);\n  });\n\n  it(\"removeCartItem
removes the line, returns true on hit\", () => {\n    addCartItem(PILOT, \"m-paracetamol-500\",
1);\n    expect(removeCartItem(PILOT, \"m-paracetamol-500\")).toBe(true);\n
expect(listCartItems(PILOT)).toEqual([]);\n  });\n\n  it(\"createOrdersFromCart fans out one
Order per distinct distributorId\", () => {\n    addCartItem(PILOT, \"m-paracetamol-500\", 2); //
d-mediplus\n    addCartItem(PILOT, \"m-vitd3-60k\", 1); // d-omsai\n    addCartItem(PILOT, \"m-
amoxicillin-500\", 1); // d-mediplus again\n    const created = createOrdersFromCart(PILOT);\n
expect(created.length).toBe(2);\n    const distributorIds = new Set(created.map((c) =>
c.distributorId));\n    expect(distributorIds).toEqual(new Set([\"d-mediplus\", \"d-omsai\"]));
\n    for (const c of created) {\n      const order = getOrder(c.id);\n
expect(order?.status).toBe(\"placed\");\n      const items = listOrderItems(c.id);\n
expect(items.length).toBeGreaterThan(0);\n      const sum = items.reduce((acc, x) => acc +
x.lineTotalPaise, 0);\n      expect(sum).toBe(order?.totalPaise);\n    }\n
expect(listCartItems(PILOT)).toEqual([]);\n  });\n\n  it(\"createOrdersFromCart returns [] when
the cart is empty\", () => {\n    expect(createOrdersFromCart(PILOT)).toEqual([]);\n  });\n});\n"
    }
  ],
  "notes": [
    "Money is stored as integer paise everywhere; the only formatting helper (paiseToRupees)
lives in lib/types.ts so it can be imported from both Route Handlers (server) and pages (client/
server). Two-decimal padding + Indian numbering match the audience's expectation.",
    "All Map instances live on globalThis under a single namespaced key (__pharmacyB2BStore).
Next.js dev mode loads each Route Handler in its own module graph, which means a plain `const x =
new Map()` at module scope produces a FRESH Map per handler - the login handler and the search
handler then see different stores. Hoisting onto globalThis is the canonical Next 14 pattern for
shared in-memory state across all server modules in one process; without it, /api/auth/login
finds the retailer but /api/medicines/search returns []. The seeded flag also lives inside the
globalThis-backed object so Next's hot reload + Route Handler re-imports do not produce duplicate
rows.",
    "The pilot retailer license number 'MH-RP-2024-7821' deliberately matches the default value
pre-filled by the t-shell-1 login page so a fresh `npm run dev` works end-to-end on the first
try.",
    "createOrdersFromCart fans out one Order per distinct distributorId, copies CartItem rows
into OrderItem rows with frozen unit prices, and clears the cart atomically (within the
synchronous block) - mirrors the contract POST /api/orders is documented to honour."
  ]
}
```

QA

Decision: PASS
Test plan: 12 item(s)
Results: 12 pass, 0 fail
Bugs: 0

BUGS (0)

(none)

FULL QA OUTPUT (JSON)


```
{
  "taskId": "t-data-1",
  "testPlan": [
    "Verify the file set covers the data spine: lib/types.ts (shared types + helpers), lib/db/
store.ts (in-memory tables, public read/write API, seed).",
    "Verify lib/types.ts exposes every interface the CTO databaseSchema implies (Retailer,
Distributor, Medicine, Offer, CartItem, Order, OrderItem, OutstandingPayment) and the supporting
OrderStatus union + ACTIVE_STATUSES / CLOSED_STATUSES / STATUS_LABELS lookups.",
    "Verify the OrderStatus taxonomy is exhaustive: every status appears in either
ACTIVE_STATUSES or CLOSED_STATUSES (not both, not neither) and has a STATUS_LABELS entry.",
    "Verify paiseToRupees handles zero, normal values, large values (Indian grouping
1,23,45,678), negatives.",
    "Verify lib/db/store.ts seeds 1 retailer, 4 distributors, 20 medicines (5 per distributor), 3
offers, 2 outstanding rows, 2 pre-existing orders (one active, one closed).",
    "Verify the pilot retailer license number is MH-RP-2024-7821 (matches the t-shell-1 login
default).",
    "Verify findRetailerByLicense is case-insensitive (the login handler doesn't normalise case
before lookup).",
    "Verify searchMedicines is case-insensitive over name + brand + generic and that an empty
query returns the full catalogue (used by the search results page when q is missing).",
    "Verify addCartItem is idempotent on (retailerId, medicineId): a second call with qty=N adds
to the existing line rather than creating a duplicate row.",
    "Verify createOrdersFromCart fans out one Order per distinct distributorId, copies CartItem
rows into OrderItem rows with frozen unit prices, totals match across both, and the cart is
cleared atomically on success.",
    "Verify the seed-guard symbol prevents double-seeding under Next's hot reload (Symbol.for +
globalThis flag).",
    "Search files[].content for forbidden placeholder strings (TODO, FIXME, XXX, 'not
implemented')."
  ],
  "results": [
    {
      "test": "File set covers the data spine.",
      "status": "pass",
      "details": "lib/types.ts (~3.2 KB), lib/db/store.ts (~12 KB). Both repo-relative, forward-
slashed."
    },
    {
      "test": "lib/types.ts ships every interface the CTO databaseSchema implies.",
      "status": "pass",
      "details": "Retailer (11 fields), Distributor (8), Medicine (12 incl. distributorId fk +
paise integers), Offer (8), CartItem (7 incl. distributorId denorm), Order (8), OrderItem (6),
OutstandingPayment (5). Plus the OrderStatus / OrderStatusGroup unions, ACTIVE_STATUSES /
CLOSED_STATUSES Sets, and STATUS_LABELS map."
    },
    {
      "test": "OrderStatus taxonomy is exhaustive and disjoint.",
      "status": "pass",
      "details": "ACTIVE_STATUSES = {placed, acknowledged, out_for_delivery}. CLOSED_STATUSES =
{delivered, cancelled}. Union covers every OrderStatus exactly once. Co-located vitest asserts
(a) for every status, (inActive XOR inClosed), and (b) STATUS_LABELS has an entry."
    },
    {
      "test": "paiseToRupees output shape.",
      "status": "pass",
      "details": "0 -> Rs.0.00; 1234567 -> Rs.12,345.67 (Indian grouping); 100000000 ->
Rs.10,00,000.00; -150 -> -Rs.1.50. Test assertions use \\u20B9 (rupee sign) literal so they
survive JSON escaping. Decimal pad always 2 digits."
    },
```

```
    {
      "test": "Seed populates all the documented rows.",
      "status": "pass",
      "details": "After import: retailers=1 (r-pilot-1), distributors=4 (d-mediplus, d-omsai, d-aakash, d-pharmacare), medicines=20 (5 per distributor), offers=3 (o-1, o-2, o-3), outstanding=2 (op-1=Rs.14,250 + op-2=Rs.8,750), orders=2 (ord-seed-1 active out_for_delivery, ord-seed-2 closed delivered). Pilot favourites = [m-paracetamol-500, m-pantoprazole-40, m-vitd3-60k]."
    },
    {
      "test": "Pilot license number matches the t-shell-1 login default.",
      "status": "pass",
      "details": "Seeded license = 'MH-RP-2024-7821'. t-shell-1 app/page.tsx pre-fills the same string. Cross-task contract preserved."
    },
    {
      "test": "findRetailerByLicense is case-insensitive.",
      "status": "pass",
      "details": "Implementation: r.licenseNumber.toLowerCase() === licenseNumber.toLowerCase(). Co-located vitest covers both 'MH-RP-2024-7821' and 'mh-rp-2024-7821'. Important because the login handler does NOT pre-normalise case."
    },
    {
      "test": "searchMedicines is case-insensitive and returns all rows on empty.",
      "status": "pass",
      "details": "q.trim().toLowerCase() applied; empty q short-circuits to the full catalogue sorted by name. Substring match across name + brand + genericName so 'paracet' / 'crocin' / 'acetaminophen' all hit m-paracetamol-500."
    },
    {
      "test": "addCartItem is idempotent on (retailerId, medicineId).",
      "status": "pass",
      "details": "Map key = `${retailerId}|${medicineId}`. existing.qty += qty when present; otherwise new CartItem with distributorId denormalised from the medicine. Test: 1 + 2 -> single line with qty 3."
    },
    {
      "test": "createOrdersFromCart fan-out invariants.",
      "status": "pass",
      "details": "Groups cart items by distributorId, creates one Order per group with totalPaise = sum(qty * unitPricePaise) and itemCount = sum(qty). OrderItem rows are deep-copied with frozen unitPricePaise + lineTotalPaise. clearCart(retailerId) called at the end, atomically within the synchronous block. Test verifies 3-line cart split across d-mediplus + d-omsai produces 2 orders, totals match across Order.totalPaise vs sum(OrderItem.lineTotalPaise), cart is empty after."
    },
    {
      "test": "Seed-guard prevents double-seeding under hot reload.",
      "status": "pass",
      "details": "const SEEDED = Symbol.for('pharmacy-b2b.store.seeded'); if (!globalThis[SEEDED]) { seed(); globalThis[SEEDED] = true; }. Symbol.for is process-wide, so re-importing store.ts via Next's hot module reload does not re-seed and double-count rows."
    },
    {
      "test": "No forbidden placeholder strings.",
      "status": "pass",
      "details": "Scanned both files for TODO, FIXME, XXX, 'not implemented'. Zero matches."
    }
  ],
  "bugs": [],
  "decision": "PASS"
}
```

Cybersecurity

Decision: GO

Summary: Audited the in-memory data spine: shared TypeScript types and the Map-backed store with seed data. The change introduces no network I/O, no external SDK calls, no LLM-mediated flow, no shell or filesystem access, and no public surface (every consumer is a Route Handler that gates access via the session middleware from t-shell-2). No critical or high-severity vulnerabilities were found. Two medium-severity items (PII at rest in the seeded retailer row, hardcoded GSTIN in source) and one low-severity item (no rate-limit on searchMedicines linear scan) are documented for the cycle-2 backlog. Nothing here blocks proceed.

Prompt-injection risk: None. The data spine ships no LLM-mediated I/O. Every public function takes typed parameters (string ids, integer qty, plain string queries) and returns typed Records. No string is ever interpolated into a model prompt, rendered as raw HTML, passed to a shell, or written to disk. Search input flows only into JS .toLowerCase() + .includes() checks against in-memory rows; there is no path through which untrusted text reaches a model surface.

VULNERABILITIES (3)

MEDIUM lib/db/store.ts ships seed data containing what looks like a real retailer's PII (name, owner, license number, store address, phone, email, GSTIN) hardcoded into the source file. If this file is ever committed to a public repository or surfaces in build artefacts (Next.js bundles server modules into .next/server/), the pilot retailer's contact details are exfiltrated.

- Recommendation: Treat the pilot retailer block as fixture-only for the demo and replace with placeholder data (e.g. 'Demo Pharmacy', '+91 00000 00000', 'demo@example.com', license 'DEMO-0001') before any public push or production build. For the actual pilot, move the retailer row to environment-loaded JSON (PHARMACY_PILOT_JSON_PATH) and load lazily on first request.

MEDIUM Money handling uses plain JS numbers for paise. JS Number is a double; integer math is exact only up to 2^53 - 1 paise (~Rs.90 trillion), which is safe for this app, but multiplication / addition without explicit overflow guards in createOrdersFromCart could silently lose precision once a single line item crosses ~Rs.18,014,398,509,481.98 (extreme but worth surfacing for a payments-adjacent surface).

- Recommendation: Add a defensive Number.isSafeInteger(...) assertion inside createOrdersFromCart for both qty * unitPricePaise and the running totalPaise sum, throwing a hard error with the offending row id if the invariant breaks. Cycle 2 hardening: migrate paise math to bigint when Postgres + Drizzle land.

LOW searchMedicines does a linear scan over Array.from(medicines.values()) on every call with toLowerCase() on three fields per row. With 20 medicines this is ~60 tolower calls per query (microseconds) and is totally fine for the pilot, but scales linearly. A retailer who scripts the search endpoint can drive ~1000 QPS without effort because there is no rate limit on the API.

- Recommendation: Cycle 2 hardening: add the same upstash/ratelimit envelope recommended for the login endpoint (e.g. 60 RPS per session), and migrate searchMedicines to a Postgres trigram index when the catalogue crosses ~1000 SKUs. Both changes are out of scope for cycle 1.

REQUIRED FIXES (0)

(none)

FULL CYBERSECURITY OUTPUT (JSON)

```
{
  "taskId": "t-data-1",
  "summary": "Audited the in-memory data spine: shared TypeScript types and the Map-backed store
with seed data. The change introduces no network I/O, no external SDK calls, no LLM-mediated
flow, no shell or filesystem access, and no public surface (every consumer is a Route Handler
that gates access via the session middleware from t-shell-2). No critical or high-severity
vulnerabilities were found. Two medium-severity items (PII at rest in the seeded retailer row,
hardcoded GSTIN in source) and one low-severity item (no rate-limit on searchMedicines linear
scan) are documented for the cycle-2 backlog. Nothing here blocks proceed.",
  "vulnerabilities": [
    {
      "severity": "medium",
      "description": "lib/db/store.ts ships seed data containing what looks like a real
retailer's PII (name, owner, license number, store address, phone, email, GSTIN) hardcoded into
the source file. If this file is ever committed to a public repository or surfaces in build
artefacts (Next.js bundles server modules into .next/server/), the pilot retailer's contact
details are exfiltrated.",
      "recommendation": "Treat the pilot retailer block as fixture-only for the demo and replace
with placeholder data (e.g. 'Demo Pharmacy', '+91 00000 00000', 'demo@example.com', license
'DEMO-0001') before any public push or production build. For the actual pilot, move the retailer
row to environment-loaded JSON (PHARMACY_PILOT_JSON_PATH) and load lazily on first request."
    },
    {
      "severity": "medium",
      "description": "Money handling uses plain JS numbers for paise. JS Number is a double;
integer math is exact only up to 2^53 - 1 paise (~Rs.90 trillion), which is safe for this app,
but multiplication / addition without explicit overflow guards in createOrdersFromCart could
silently lose precision once a single line item crosses ~Rs.18,014,398,509,481.98 (extreme but
worth surfacing for a payments-adjacent surface).",
      "recommendation": "Add a defensive Number.isSafeInteger(...) assertion inside
createOrdersFromCart for both qty * unitPricePaise and the running totalPaise sum, throwing a
hard error with the offending row id if the invariant breaks. Cycle 2 hardening: migrate paise
math to bigint when Postgres + Drizzle land."
    },
    {
      "severity": "low",
      "description": "searchMedicines does a linear scan over Array.from(medicines.values()) on
every call with toLowerCase() on three fields per row. With 20 medicines this is ~60 tolower
calls per query (microseconds) and is totally fine for the pilot, but scales linearly. A retailer
who scripts the search endpoint can drive ~1000 QPS without effort because there is no rate limit
on the API.",
      "recommendation": "Cycle 2 hardening: add the same upstash/ratelimit envelope recommended
for the login endpoint (e.g. 60 RPS per session), and migrate searchMedicines to a Postgres
trigram index when the catalogue crosses ~1000 SKUs. Both changes are out of scope for cycle 1."
    }
  ],
  "promptInjectionRisk": "None. The data spine ships no LLM-mediated I/O. Every public function
takes typed parameters (string ids, integer qty, plain string queries) and returns typed Records.
No string is ever interpolated into a model prompt, rendered as raw HTML, passed to a shell, or
written to disk. Search input flows only into JS .toLowerCase() + .includes() checks against in-
memory rows; there is no path through which untrusted text reaches a model surface.",
  "decision": "GO",
  "requiredFixes": []
}
```